SPECIAL SESSION · 7 SLIDES · BANKING & FINANCE

Memory Engineering

Context engineering decides what enters one call. Memory engineering decides what survives after that call ends, across every thread a customer will ever open.

Memory Engineering | Store vs Checkpointer | Write Policy | Semantic · Episodic · Procedural | Identity Scoping

KGP Talkie
LEARN. BUILD. TALK AI

Scan to subscribe

Type of Agentic AI Engineering

KGP Talkie
LEARN. BUILD. TALK AI

Prompt
What do I say?

Context
What goes in the window, what stays out?

Memory
What survives once this thread ends?

FURTHEST REACH

Harness
What does the code around the model do?

Loop
How many calls, and what says the work is done?

Graph
How do several loops coordinate?

WHY IT REACHES FURTHEST

customer A → customer B

LEAKS

The other five fail inside one conversation.

Memory does not.

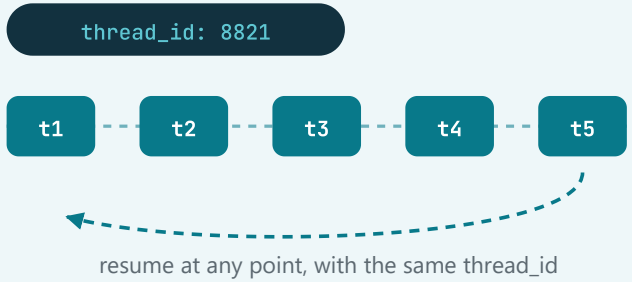
WIDTH = REACH how far a mistake in that layer can travel, whatever order you meet it in

One Thread, or Every Thread This Customer Has

LOOKS LIKE a database with two tables, one per chat, one per customer.

It kind of is. The line is what survives the thread closing.

CHECKPOINTER · ONE THREAD



A new thread_id tomorrow, and it knows **nothing about this customer.**

STORE · EVERY THREAD



Written in thread one, read back in thread nine, weeks later.

One namespace per customer, not per conversation.

Loop engineering's window versus store lived inside one thread.

This line is one thread, versus every thread this customer will ever open.

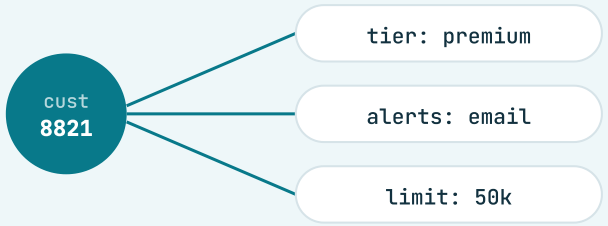
Three Kinds of Remembering

LOOKS LIKE one big memory blob, everything the agent has ever seen.

Three shapes. Mixing them is why retrieval comes back vague.

Semantic

a fact, still true next week

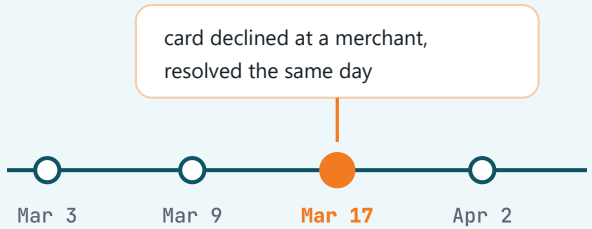


BACKED BY

STORE — one key per fact

Episodic

an event, the whole story

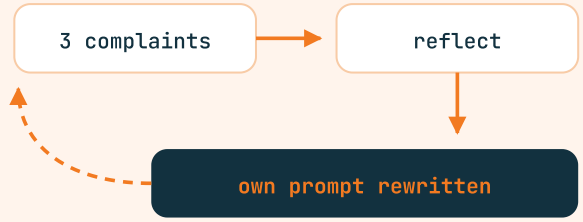


BACKED BY

CHECKPOINTED THREADS — kept whole

Procedural

a rule it edits about itself



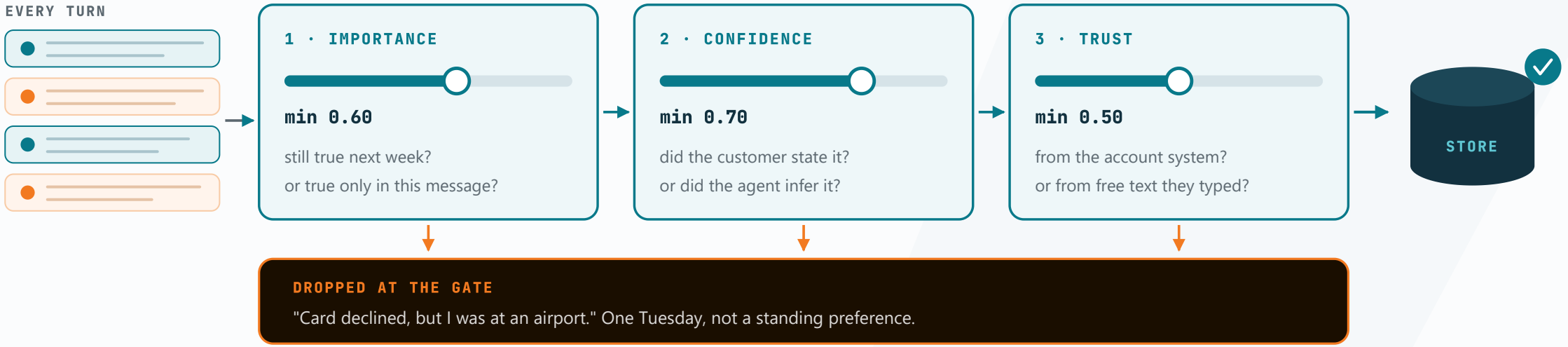
BACKED BY

STORE — the prompt itself

Retrieval Is Only as Good as What Got Written

LOOKS LIKE a small detail, decided later, after retrieval is built.

It is the first decision. Most systems skip it.



The single biggest lever on retrieval quality is not the search index.

It is the write policy sitting upstream of it.

Two Stores, and What Still Goes Wrong

PYTHON · LANGCHAIN 1.X · STORE AND CHECKPOINTER

```python
from langchain.agents import create_agent
from langgraph.store.memory import InMemoryStore
from langgraph.checkpoint.memory import InMemorySaver

store = InMemoryStore()          # long-term, keyed by namespace
checkpointer = InMemorySaver()   # short-term, keyed by thread_id

agent = create_agent(
    model=model,
    tools=[get_statement, update_preference],
    store=store,
    checkpointer=checkpointer,
)

# every write goes through one gate, never straight to the store
def remember(store, customer_id, fact, importance, confidence, trust):
    if importance ⩾ 0.6 and confidence ⩾ 0.7 and trust ⩾ 0.5:
        namespace = ("customer", customer_id)  # identity, not convenience
        store.put(namespace, key=fact["key"], value=fact)

# swap InMemoryStore for PostgresStore in production. Same three calls.
```

FOUR WAYS MEMORY FAILS

It leaks across customers
B hears what only A ever said.
CAUGHT BY namespace from the session, not the model

It never goes stale
A phone number nobody uses now.
CAUGHT BY a TTL, checked on every read

It doubles back on itself
One preference, saved four ways.
CAUGHT BY dedup before the write, not cleanup after

It was never true
One sarcastic line, kept forever.
NO BRAKE re-read what low-confidence writes did

Four Things to Keep

KGP Talkie
LEARN. BUILD. TALK AI

Two different lifetimes

Checkpointer for the thread.
Store for the customer.

Name the kind first

A fact, an event, a rule.
Three shapes, not one blob.

Gate the write

Importance, confidence, trust.
Not just a similarity score.

Scope by identity

Namespace from the session.
Never from a typed field.

Does this still need to be true next week?

NO, LEAVE IT IN THE THREAD
The checkpointer already covers it. No store needed.

YES, IT BELONGS IN THE STORE
Name its kind, gate the write, scope it to one identity.